

# Assignment 2

## FPGA Design Practices

Tianze Zhang  
1005459585

# 1 Random Access Memories (RAMs)

Q1. A table listing the resource utilization obtained for each implementation

| Ram_style       | Ram size | Slice LUTs | Slice Registers | Block RAM |
|-----------------|----------|------------|-----------------|-----------|
| Registers       | 256x32   | 4616       | 8256            | NA        |
| Distributed RAM | 256x32   | 420        | 64              | NA        |
| Block RAM       | 256x32   | NA         | NA              | 1         |
| Registers       | 512x32   | 9331       | 16448           | NA        |
| Distributed RAM | 512x32   | 840        | 64              | NA        |
| Block RAM       | 512x32   | NA         | NA              | 1         |

Q2. A discussion comparing the different storage primitives.  
When would you want to use a specific primitive to implement a certain memory?

Registers: Register access is extremely rapid, taking only one clock cycle. However, Registers use a high number of slice registers in the FPGA fabric, limiting their scalability for bigger memory. Registers are often constructed using flip-flops or latches and are best suited for activities that demand rapid processing.

Distributed RAM is accomplished with a mix of LUTs and flip-flops, resulting in a balance of performance and resource utilization. It is appropriate for applications where latency requirements are not as severe as with registers, but resource utilization must be optimized.

Block RAM is a type of dedicated memory that provides high-density storage and may be accessed in a few clock cycles. It has a higher latency than the other two alternatives. It is appropriate for applications requiring minimal latency on huge data collections, such as data processing.

## 2 DSP Units

Q1. A table listing the resource utilization and Fmax obtained for each of your four implementations

| CONFIG         | DSP48E1 | LUT                   | REGISTER | FMAX(Mhz) |
|----------------|---------|-----------------------|----------|-----------|
| DSP WIDTH16    | 1       | 1(for out_vld signal) | 3        | 689       |
| NO DSP WIDTH16 | NA      | 362                   | 117      | 689       |
| DSP WIDTH32    | 5       | 49                    | 37       | 225       |
| NO DSP WIDTH32 | NA      | 1284                  | 229      | 689       |

Q2. A description of how you made your implementation pack into a single DSP48E1 block and what changed when the width was set to 32-bits

To enable the DSP block, add the following code at the start of the module: (\*use\_dsp48 = "yes"\*).

After changing the width to 32 bits. The number of DSP48E1 slices is raised to 7, and 64 LUTs are employed. The reason is because a single DSP slice cannot handle data with a width of 32 bits.

Q3. A discussion comparing the utilization and Fmax of LUT vs DSP implementations. Are there cases when you would prefer using a particular resource?

In terms of utilization, with a 32-bit data width, employing DSP costs 2.92% of total DSP consumption, but using simply FF and LUTs costs 2.1% of total utilization. As a result, the usage cost is comparable to one another. The Fmax is greater with DSP slices than without them. This is because DSP is specialized hardware that can do arithmetic operations like add and multiply

in a single cycle. On the other hand, LUTs take many clock cycles to do the same work, resulting in a lower Fmax.

If the application is primarily concerned with arithmetic operations, such as video processing, DSP is a preferable solution; if the application requires a combination of arithmetic and logic functions, LUTs are appropriate.

### 3 Resource Trade-offs

Q1. A table listing the resource utilization obtained for each implementation

| Type           | LUTs | Registers | DSPs | Mem |
|----------------|------|-----------|------|-----|
| Memory Mapping | NA   | NA        | NA   | 0.5 |
| Direct compute | 55   | NA        | 1    | NA  |

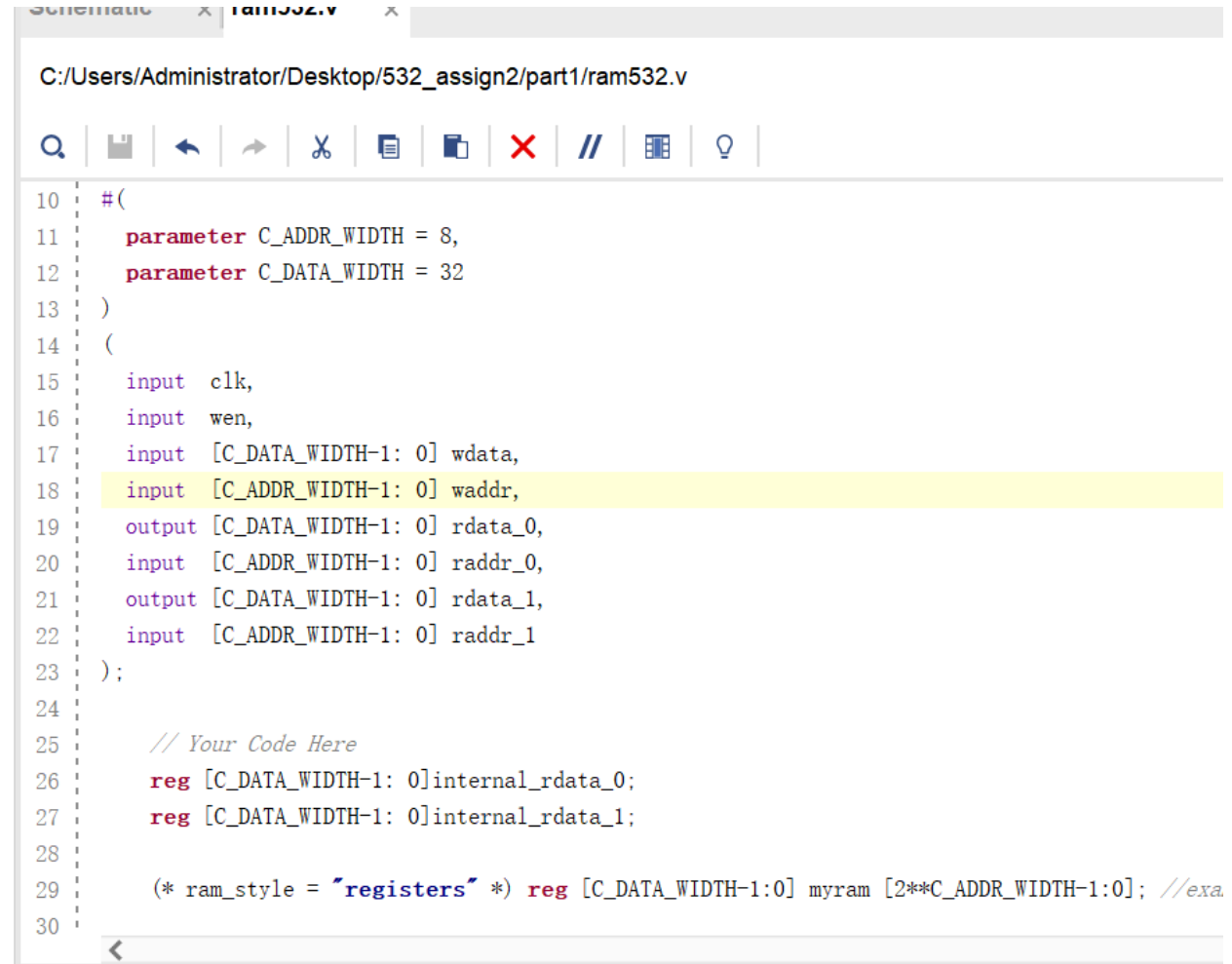
Q2. A discussion comparing the different techniques. When would it be beneficial to use a RAM LUT versus performing the computation? Is it possible to combine the two techniques?

RAM LUT is useful when dealing with vast amounts of data and repetitive computations. It is possible to combine the two methods: if an application requires repetitive computation with varied inputs and needs to recall the computed results later to perform additional actions, the computed results could be stored in a BRAM and then retrieved directly, potentially resulting in faster overall performance.

# Appendix

## PART1

### Registers with width 256



```
10  #(
11      parameter C_ADDR_WIDTH = 8,
12      parameter C_DATA_WIDTH = 32
13  )
14  (
15      input  clk,
16      input  wen,
17      input  [C_DATA_WIDTH-1: 0] wdata,
18      input  [C_ADDR_WIDTH-1: 0] waddr,
19      output [C_DATA_WIDTH-1: 0] rdata_0,
20      input  [C_ADDR_WIDTH-1: 0] raddr_0,
21      output [C_DATA_WIDTH-1: 0] rdata_1,
22      input  [C_ADDR_WIDTH-1: 0] raddr_1
23  );
24
25      // Your Code Here
26      reg [C_DATA_WIDTH-1: 0] internal_rdata_0;
27      reg [C_DATA_WIDTH-1: 0] internal_rdata_1;
28
29      (* ram_style = "registers" *) reg [C_DATA_WIDTH-1:0] myram [2**C_ADDR_WIDTH-1:0]; //exa
30
```

ports

Design Runs

Utilization

×

Debug

Q

⌵

⌴

%

Hierarchy

| Name   | ^1 | Slice LUTs<br>(63400) | Slice Registers<br>(126800) | F7 Muxes<br>(31700) | F8 Muxes<br>(15850) |  |
|--------|----|-----------------------|-----------------------------|---------------------|---------------------|--|
| ram532 |    | 4616                  | 8256                        | 2176                | 1088                |  |

Distributed with width 256

Schematic x ram532.v x

C:/Users/Administrator/Desktop/532\_assign2/part1/ram532.v

Q | | | | | | | | | | | |

```
10  #(
11      parameter C_ADDR_WIDTH = 8,
12      parameter C_DATA_WIDTH = 32
13  )
14  (
15      input  clk,
16      input  wen,
17      input  [C_DATA_WIDTH-1: 0] wdata,
18      input  [C_ADDR_WIDTH-1: 0] waddr,
19      output [C_DATA_WIDTH-1: 0] rdata_0,
20      input  [C_ADDR_WIDTH-1: 0] raddr_0,
21      output [C_DATA_WIDTH-1: 0] rdata_1,
22      input  [C_ADDR_WIDTH-1: 0] raddr_1
23  );
24
25      // Your Code Here
26      reg [C_DATA_WIDTH-1: 0] internal_rdata_0;
27      reg [C_DATA_WIDTH-1: 0] internal_rdata_1;
28
29      (* ram_style = "distributed" *) reg [C_DATA_WIDTH-1:0] myram [2**C_ADDR_WIDTH-1:0]; //
30
```

Design Runs Utilization x Debug

Q | | | | | Hierarchy

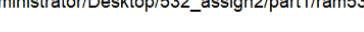
| Name   | 1 | Slice LUTs<br>(63400) | Slice Registers<br>(126800) |
|--------|---|-----------------------|-----------------------------|
| ram532 |   | 420                   | 64                          |



Distributed with width 512:

Schematic x ram532.v x

C:/Users/Administrator/Desktop/532\_assign2/part1/ram532.v



```
10 #(
11     parameter C_ADDR_WIDTH = 9,
12     parameter C_DATA_WIDTH = 32
13 )
14 (
15     input clk,
16     input wen,
17     input [C_DATA_WIDTH-1: 0] wdata,
18     input [C_ADDR_WIDTH-1: 0] waddr,
19     output [C_DATA_WIDTH-1: 0] rdata_0,
20     input [C_ADDR_WIDTH-1: 0] raddr_0,
21     output [C_DATA_WIDTH-1: 0] rdata_1,
22     input [C_ADDR_WIDTH-1: 0] raddr_1
23 );
24
25 // Your Code Here
26 reg [C_DATA_WIDTH-1: 0] internal_rdata_0;
27 reg [C_DATA_WIDTH-1: 0] internal_rdata_1;
28
29 (* ram_style = "distributed" *) reg [C_DATA_WIDTH-1:0] myram [2**C_ADDR_WIDTH-1:0]; //example code from AMD website.
```

| ports  | Design Runs           | Utilization                 | x                   | Debug     |
|--------|-----------------------|-----------------------------|---------------------|-----------|
| Q      | ≡                     | ⚙                           | %                   | Hierarchy |
| Name   | Slice LUTs<br>(63400) | Slice Registers<br>(126800) | F7 Muxes<br>(31700) |           |
| ram532 | 840                   | 64                          | 64                  |           |

Block with width 512:

```

25 // Your Code Here
26 reg internal_rdata_0;
27 reg internal_rdata_1;
28
29 (* ram_style = "block" *) reg [C_DATA_WIDTH-1:0] myram [2**C_ADDR_WIDTH-1:0]; //example code from AMD website.
30
31 always @ (posedge clk) begin
32     if (wen) begin //When write enable is high
33         myram[waddr] <= wdata; //the data contained in the wdata wires is to be stored in the RAM at address waddr in the next clock cycle

```

|           |                      |             |       |
|-----------|----------------------|-------------|-------|
| Reports   | Design Runs          | Utilization | Debug |
| Hierarchy |                      |             |       |
| Name      | Block RAM Tile (135) |             |       |
| ram532    | 1                    |             |       |

# PART2

## Single DSP with width of 16

Schematicxpreadd532.vxtest\_2.vxSchematic (2)x

5 Cells84 I/O Ports90 Nets

The schematic shows a single DSP48E1 block. Inputs in\_a[15:0] and in\_b[15:0] are connected to the A and B inputs of the DSP. Input in\_a[15:0] clk is connected to the CLK pin. Input in\_vld is connected to the VLD pin. The DSP's output out[32:0] is connected to the out\_vld output. The circuit also includes two FDRE registers (in\_vld\_delay\_reg and inter\_out\_vld\_delay\_reg) and a LUT3 block (internal\_out\_vld\_l\_1) used for signal conditioning and delay.

ReportsDesign RunsUtilizationxDebug

Hierarchy

| Name      | Slice LUTs (63400) | Slice Registers (126800) | DSPs (240) |
|-----------|--------------------|--------------------------|------------|
| preadd532 | 1                  | 3                        | 1          |

preadd532

Nets (90)  
Leaf Cells (8)

Properties

Select an object to see properties

1 (\* use\_dsp48 = "yes" \*) // set to yes in order to using DSP block

2 module preadd532

3 {

4 parameter C\_WIDTH = 16

5 }

6 (

7 input clk,

8 input in\_vld,

9 )

Tcl ConsoleMessagesLogReportsDesign RunsTimingxUtilizationDebug

Unconstrained Paths - clk - NONE - Setup

| Name    | Slack | Levels | Routes | High Fanout | From                   | To      | Total Delay | Logic Delay | Net Delay | Requirement | Source Clock | Destination Clock | Exception |
|---------|-------|--------|--------|-------------|------------------------|---------|-------------|-------------|-----------|-------------|--------------|-------------------|-----------|
| Path 27 | ∞     | 0      | 1      | 1           | internal_out_vld_reg/C | out_vld | 1.451       | 0.478       | 0.973     | ∞           | clk          | Destination Clock |           |
| Path 28 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[0]  | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 29 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[10] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 30 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[11] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 31 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[12] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 32 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[13] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 33 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[14] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 34 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[15] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 35 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[16] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |
| Path 36 | ∞     | 0      | 1      | 0           | m_reg_reg/CLK          | out[17] | 1.407       | 0.434       | 0.973     | ∞           | clk          |                   |           |

No DSP width 16:

Schematic x preadd532.v x test\_2.v x Schematic (2) x

C:/Users/Administrator/Desktop/532\_assign2/part2/preadd532.v

Q | Save | Undo | Redo | Cut | Copy | Paste | Delete | Comment | Help

```
1 (* use_dsp48 = "no" *) // set to yes in order to using DSP block
2 module preadd532
3 #
4     parameter C_WIDTH = 16
5 )
6 (
7     input clk,
8     input in_vld,
```

ports | Design Runs | Timing | Utilization x | Debug

Q | Zoom In | Zoom Out | % | Hierarchy

| Name      | Slice LUTs<br>(63400) | Slice Registers<br>(126800) |
|-----------|-----------------------|-----------------------------|
| preadd532 | 362                   | 117                         |

## DSP 32 with width 32:

The screenshot displays the Xilinx Vivado IDE interface. The top tab bar shows 'Schematic', 'preadd532.v', 'test\_2.v', and 'Schematic (2)'. The main editor window shows the Verilog code for 'preadd532.v' located at 'C:/Users/Administrator/Desktop/532\_assign2/part2/preadd532.v'. The code defines a module with a parameter 'C\_WIDTH = 32' and inputs/outputs for clock, validity, and data. The bottom panel shows the 'Utilization' report, which includes a table of resource usage for the 'preadd532' module.

```
1 (* use_dsp48 = "yes" *) // set to yes in order to using DSP block
2 module preadd532
3 #
4     parameter C_WIDTH = 32
5 )
6 (
7     input clk,
8     input in_vld,
9     input signed [C_WIDTH-1: 0] in_a,
10    input signed [C_WIDTH-1: 0] in_b,
11    input signed [C_WIDTH-1: 0] in_c,
12    output signed [2*C_WIDTH: 0] out,
13    output out_vld
14 )
```

| Name      | Slice LUTs<br>(63400) | Slice Registers<br>(126800) | DSPs<br>(240) |
|-----------|-----------------------|-----------------------------|---------------|
| preadd532 | 49                    | 37                          | 5             |

chematic x preadd532.v x test\_2.v x Schematic (2) x

C:/Users/Administrator/Desktop/532\_assign2/part2/preadd532.v

Q

🏠

↶

↷

✂

📄

📁

✖

//

🔍

🔗

1

(\* use\_dsp48 = "yes" \*) // set to yes in order to using DSP block

2

module preadd532

3

#{

4

parameter C\_WIDTH = 32

5

}

6

(

7

input clk,

8

input in\_vld,

9

input signed [C\_WIDTH-1: 0] in\_a,

10

input signed [C\_WIDTH-1: 0] in\_b,

11

input signed [C\_WIDTH-1: 0] in\_c,

12

output signed [2\*C\_WIDTH: 0] out,

13

output out\_vld

14

)

Design Runs

Timing x

Utilization

Debug

Unconstrained Paths - clk - NONE - Setup

| Name      | Slack | Levels | Routes | High Fanout | From               | To      | Total Delay | Logic Delay | Net Delay | Requirement | Source Clock | Destination Clock | Exception | Clock |
|-----------|-------|--------|--------|-------------|--------------------|---------|-------------|-------------|-----------|-------------|--------------|-------------------|-----------|-------|
| ↳ Path 41 | ∞     | 13     | 14     | 1           | m_reg_reg[16]__0/C | out[64] | 4.444       | 2.805       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 42 | ∞     | 12     | 13     | 1           | m_reg_reg[16]__0/C | out[61] | 4.432       | 2.793       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 43 | ∞     | 12     | 13     | 1           | m_reg_reg[16]__0/C | out[63] | 4.426       | 2.787       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 44 | ∞     | 12     | 13     | 1           | m_reg_reg[16]__0/C | out[62] | 4.351       | 2.712       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 45 | ∞     | 12     | 13     | 1           | m_reg_reg[16]__0/C | out[60] | 4.327       | 2.688       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 46 | ∞     | 11     | 12     | 1           | m_reg_reg[16]__0/C | out[57] | 4.315       | 2.676       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 47 | ∞     | 11     | 12     | 1           | m_reg_reg[16]__0/C | out[59] | 4.309       | 2.670       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 48 | ∞     | 11     | 12     | 1           | m_reg_reg[16]__0/C | out[58] | 4.234       | 2.595       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 49 | ∞     | 11     | 12     | 1           | m_reg_reg[16]__0/C | out[56] | 4.210       | 2.571       | 1.639     | ∞           | clk          |                   |           |       |
| ↳ Path 50 | ∞     | 10     | 11     | 1           | m_reg_reg[16]__0/C | out[53] | 4.198       | 2.559       | 1.639     | ∞           | clk          |                   |           |       |

## NO DSP WITH width 32:

```
1 (* use_dsp48 = "no" *) // set to yes in order to using DSP block
2 module preadd532
3 #
4     parameter C_WIDTH = 32
5 )
6 (
7     input clk,
8     input in_vld,
9     input signed [C_WIDTH-1: 0] in_a,
10    input signed [C_WIDTH-1: 0] in_b,
11    input signed [C_WIDTH-1: 0] in_c,
12    output signed [2*C_WIDTH: 0] out,
13    output out_vld
14 )
```

Below the code editor, the 'Utilization' report is displayed, showing the resource usage for the 'preadd532' module.

| Name      | Slice LUTs<br>(63400) | Slice Registers<br>(126800) |
|-----------|-----------------------|-----------------------------|
| preadd532 | 1284                  | 229                         |

Schematic
x
preadd532.v
x
test\_2.v
x
Schematic (2)
x

C:/Users/Administrator/Desktop/532\_assign2/part2/preadd532.v

1 (\* use\_dsp48 = "no" \*) // set to yes in order to using DSP block  
2 module preadd532  
3 #(  
4 parameter C\_WIDTH = 32  
5 )  
6 (  
7 input clk,  
8 input in\_vld,  
9 input signed [C\_WIDTH-1: 0] in\_a,  
10 input signed [C\_WIDTH-1: 0] in\_b,  
11 input signed [C\_WIDTH-1: 0] in\_c,  
12 output signed [2\*C\_WIDTH: 0] out,  
13 output out\_vld

ts
Design Runs
Timing
x
Utilization
Debug

Unconstrained Paths - clk - NONE - Setup

| Name      | Slack | Levels | Routes | High Fanout | From            | To      | Total Delay | Logic Delay | Net Delay | Requirement | Source Clock | Destination |
|-----------|-------|--------|--------|-------------|-----------------|---------|-------------|-------------|-----------|-------------|--------------|-------------|
| ↳ Path 41 | ∞     | 0      | 1      | 0           | m_reg_reg[0]/C  | out[0]  | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 42 | ∞     | 0      | 1      | 0           | m_reg_reg[10]/C | out[10] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 43 | ∞     | 0      | 1      | 0           | m_reg_reg[11]/C | out[11] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 44 | ∞     | 0      | 1      | 0           | m_reg_reg[12]/C | out[12] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 45 | ∞     | 0      | 1      | 0           | m_reg_reg[13]/C | out[13] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 46 | ∞     | 0      | 1      | 0           | m_reg_reg[14]/C | out[14] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 47 | ∞     | 0      | 1      | 0           | m_reg_reg[15]/C | out[15] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 48 | ∞     | 0      | 1      | 0           | m_reg_reg[16]/C | out[16] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 49 | ∞     | 0      | 1      | 0           | m_reg_reg[17]/C | out[17] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |
| ↳ Path 50 | ∞     | 0      | 1      | 0           | m_reg_reg[18]/C | out[18] | 1.451       | 0.478       | 0.973     | ∞           | clk          |             |

## PART3

Using computation:

cube532

- > Nets (173)
- > Leaf Cells (90)

Source File Property

cube532.v

Enabled

Location: C:/Users/Administrat

General Properties

```

1 module cube532
2 (
3     input clk,
4     input [7:0] in,
5     output [23:0] out
6 );
7
8 // Your Code Here
9     reg [23:0] intern_out;
10
11     always @ (posedge clk) begin
12         intern_out <= (in**3);
13     end
14
15     assign out = intern_out;
16 endmodule
17

```

Tcl Console Messages Log Reports Design Runs Utilization x Debug

Hierarchy

Summary

Slice Logic

Slice LUTs (<1%)

| Name    | Slice LUTs (63400) | DSPs (240) |
|---------|--------------------|------------|
| cube532 | 55                 | 1          |

## Using Mem file:

```

8 // Your Code Here
9 (* ram_style = "block" *) reg [23:0] myram [255:0]; //example code from AMD website.
10 reg [23:0] intern_out;
11
12 // Initialize BRAM with cube values from a .mem file
13 initial begin
14     $readmemh("cube.mem", myram);
15 end
16
17 always @ (posedge clk) begin
18     intern_out <= myram[in];
19 end
20
21 assign out = intern_out;
22 endmodule
23

```

ports Design Runs Utilization x Debug

Hierarchy

| Name    | Block RAM Tile (135) |
|---------|----------------------|
| cube532 | 0.5                  |